

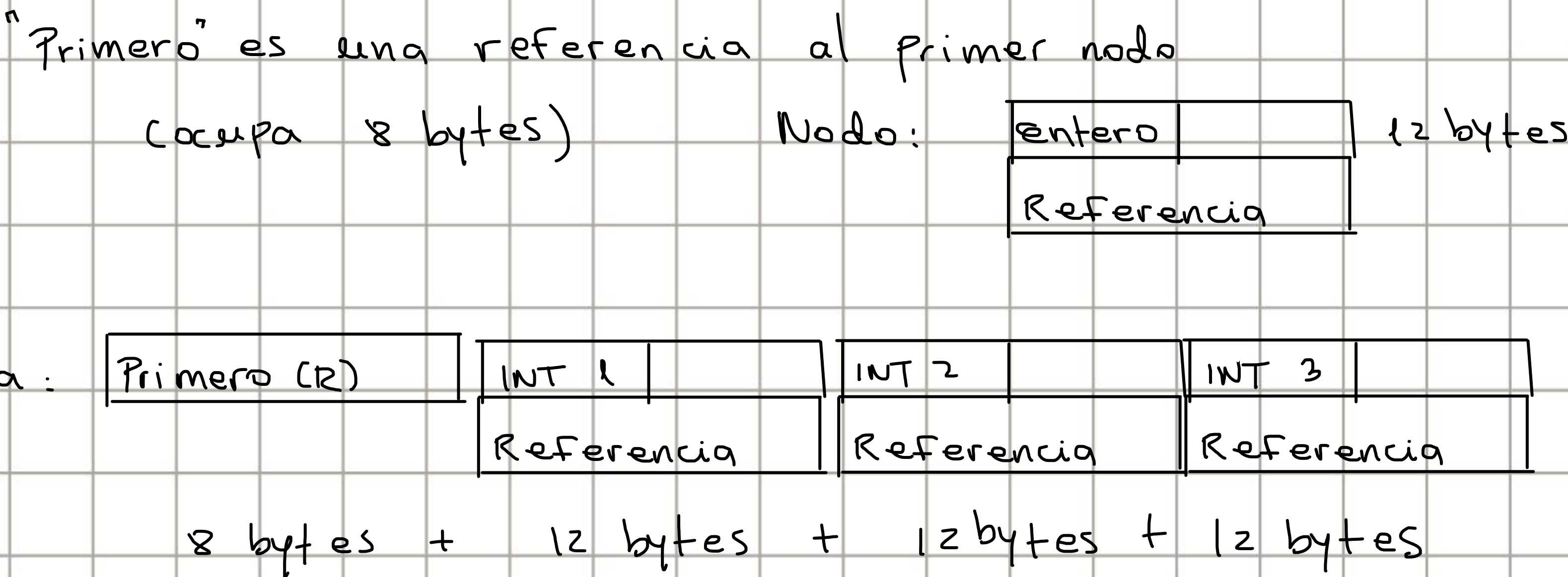
LISTAS

Opción Múltiple

1. Se define una clase **Lista de Enteros** de la siguiente manera:

```
public class Lista {  
    private Nodo primero;  
    // Métodos  
}
```

Donde la clase **Nodo** contiene un entero y una referencia al nodo siguiente. Si cada entero ocupa **4 bytes** y cada referencia ocupa **8 bytes**, entonces, **un objeto de la clase Lista** al que se le ingresaron 3 enteros ocupará:
a. Ninguna de las otras es correcta
b. 8 bytes
c. 44 bytes
d. 36 bytes

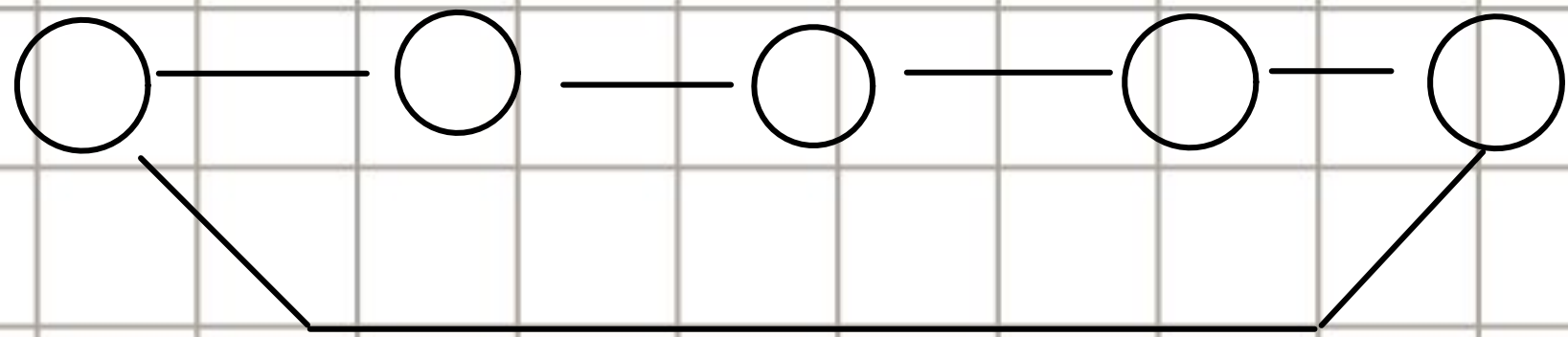


Verdadero / Falso

- 1. El TDA Lista puede implementarse en un vector.
Verdadero
Falso
- 2. Cuando se define una clase Pila, con nodos **simplemente enlazados**, conviene tener como atributo un puntero al primer nodo y al último.
Verdadero
Falso

Desarrollo: Implementación

1. Explicá cómo sería la implementación del **alta** en una **lista circular** de **máximo N nodos**. Podés escribir en la caja que está debajo o subir un archivo.

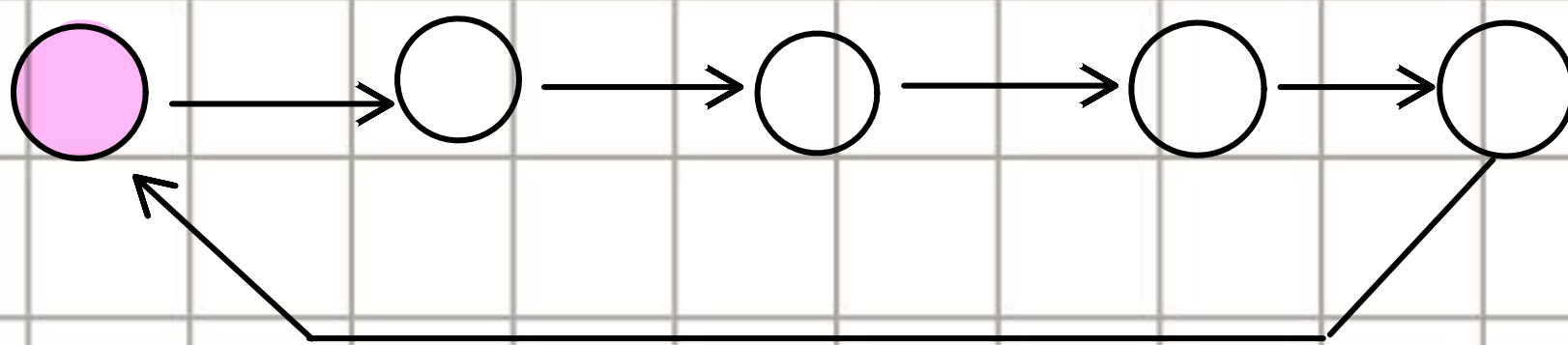


Como la lista tiene un máximo de nodos (N) depende:

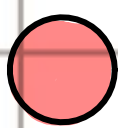
si la lista es vacía:

creamos un nuevo nodo que llamamos último

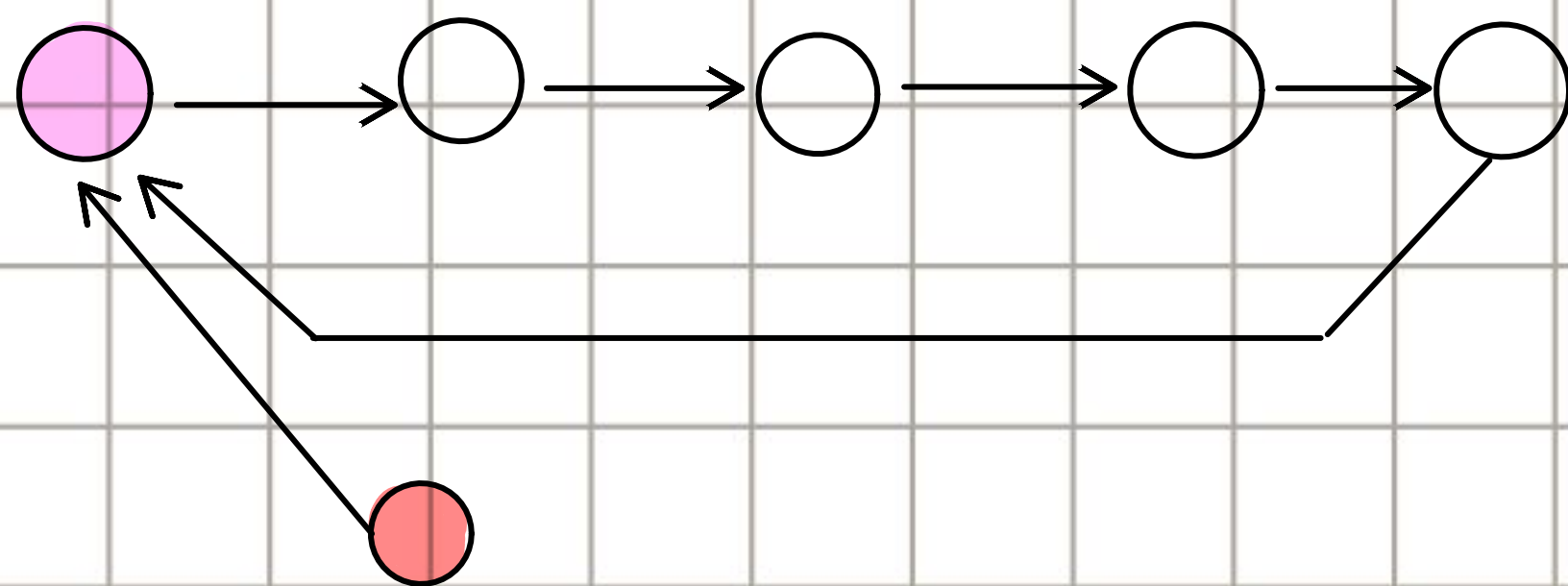
Si la # nodos $< N \Rightarrow$



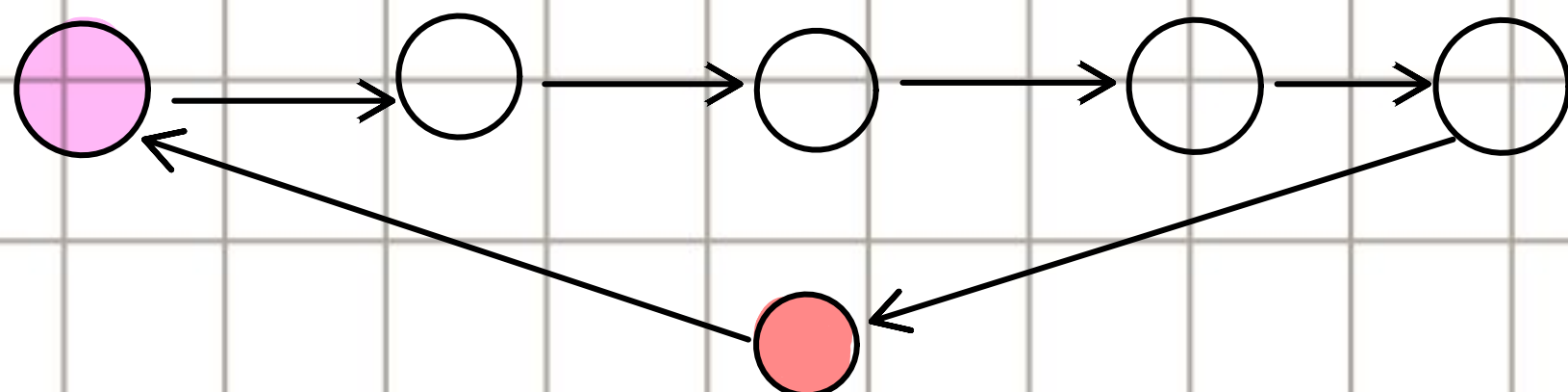
creamos un nuevo nodo



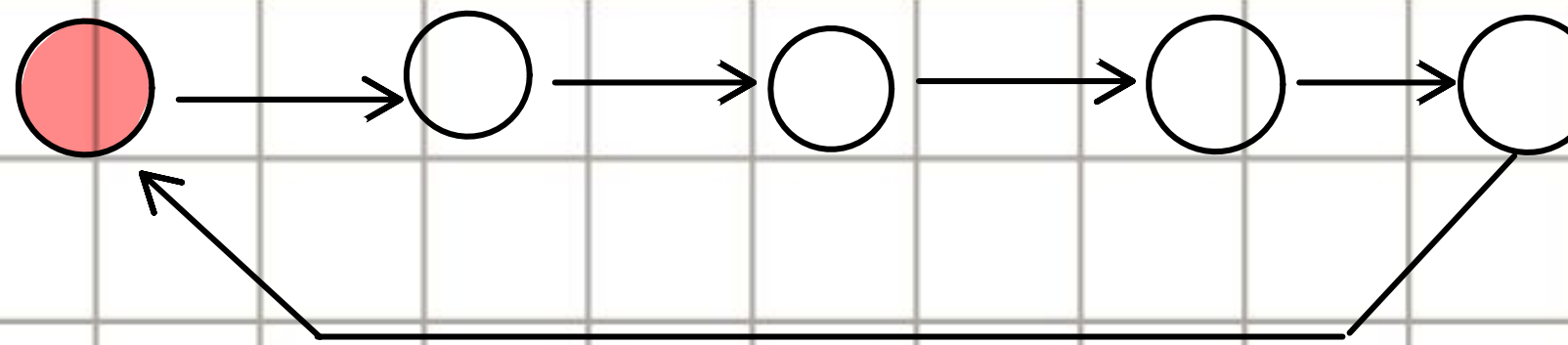
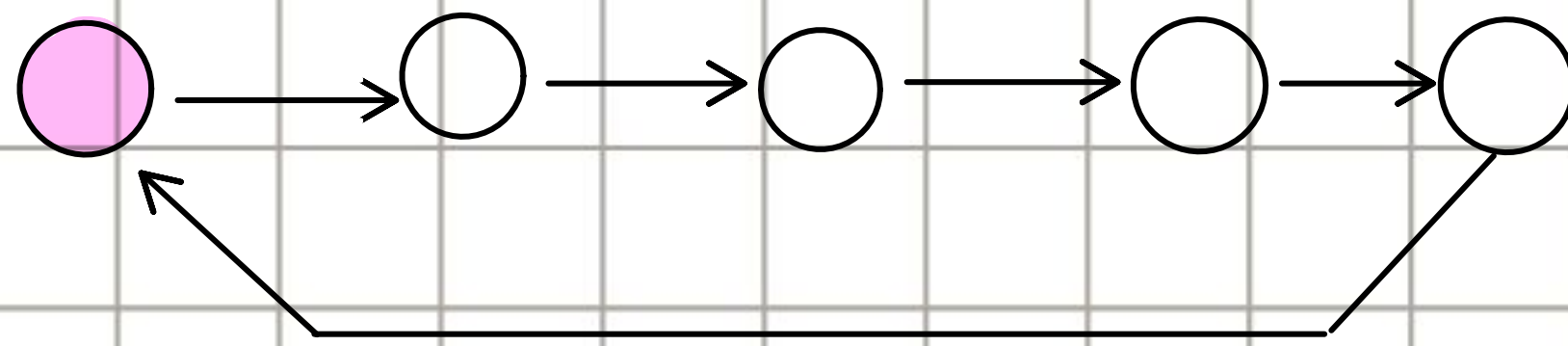
Hacemos que el nuevo nodo apunte a lo que apunta el nodo que llamamos último



y luego hacemos que el último apunte al nuevo nodo, llamando como último ahora al nuevo nodo.



Si la # nodos = N $\Rightarrow$ no necesitamos crear un nuevo nodo, simplemente reemplazamos el dato del más viejo



y lo nombramos como nuevo último.

2. Dada una lista circular de N nodos, en donde se mantienen los últimos N datos recibidos, escribí el método

```
public void alta(T dato);
```

Tené en cuenta que, si hay menos de N nodos, el alta debe ir al final del último nodo.

```

public class ListaCircularT<T> { no usages
    private NodoT ultimo; 8 usages
    private int n; 2 usages
    private int cantidad; 9 usages

    public ListaCircularT(int n) { no usages
        this.ultimo = null;
        this.n = n;
        this.cantidad = 0;
    }

    public void alta(T dato) { no usages
        if(this.cantidad == 0) {
            this.ultimo = new NodoT(dato);
            this.ultimo.cambiarSiguiente(this.ultimo);
            this.cantidad++;
        } else {
            NodoT primero = this.ultimo.obtenerSiguiente();

            if (cantidad < n) {
                NodoT nuevo = new NodoT(dato);
                nuevo.cambiarSiguiente(primero);
                ultimo.cambiarSiguiente(nuevo);
                ultimo = nuevo;
                cantidad++;
            } else {
                primero.cambiarDato(dato);
                ultimo = primero;
            }
        }
    }
}
  
```

```

private class NodoT { 17 usages
    private T dato; 3 usages
    private NodoT siguiente; 3 usages

    public NodoT(T d) { 3 usages
        this.dato = d;
        this.siguiente = null;
    }

    public T obtenerDato() { return dato; }

    public void cambiarDato(T dato) { this.dato = dato; }

    public NodoT obtenerSiguiente() { return siguiente; }

    public void cambiarSiguiente(NodoT nodo) { this.siguiente = nodo; }
}
  
```

3. Dada una lista doblemente enlazada implementá el método

```
public void alta(T dato);
```

Coloca al dato en la lista de forma de que la lista quede ordenada.

Asumí que el resto de los métodos básicos ya están implementados y que T implementa Comparable, es decir, tiene un método compareTo(Dato d) que devuelve

- Un negativo si el dato es menor al que se pasa por parámetro,
- Un positivo, si es mayor o
- 0 si son iguales.

```
public void alta(T dato) { no usages
    NodoDobleT nuevo = new NodoDobleT(dato);

    if (this.primerO == null) {
        this.primerO = nuevo;
        this.ultimo = nuevo;
    } else if (dato.compareTo(this.primerO.obtenerDato()) < 0) {
        nuevo.cambiarSiguiente(this.primerO);
        this.primerO.cambiarAnterior(nuevo);
        this.primerO = nuevo;
    } else {
        NodoDobleT actual = this.primerO;

        while (actual.obtenerSiguiente() != null && dato.compareTo(actual.obtenerSiguiente().obtenerDato()) > 0) actual = actual.obtenerSiguiente();

        NodoDobleT siguiente = actual.obtenerSiguiente();

        actual.cambiarSiguiente(nuevo);
        nuevo.cambiarAnterior(actual);

        if (siguiente != null) {
            nuevo.cambiarSiguiente(siguiete);
            siguiente.cambiarAnterior(nuevo);
        } else {
            this.ultimo = nuevo;
        }
    }
}
```


4. Dada una lista doblemente enlazada implementá el método

```
public void baja(T dato);
```

Elimina el dato de la lista junto al nodo en el que se encuentra.

Asumí que el resto de los métodos básicos ya están implementados y que T implementa Comparable, es decir, tiene un método compareTo(Dato d) que devuelve

- Un negativo si el dato es menor al que se pasa por parámetro,
- Un positivo, si es mayor o
- 0 si son iguales.

```
public void baja(T dato) { no usages
    NodoDobleT actual = this.primerO;
    while (actual != null && actual.obtenerDato().compareTo(dato) != 0) actual = actual.obtenerSiguiente();

    if(actual != null) {
        NodoDobleT anterior = actual.obtenerAnterior();
        NodoDobleT siguiente = actual.obtenerSiguiente();

        if (anterior == null) this.primerO = siguiente;
        else anterior.cambiarSiguiente(siguiente);

        if (siguiente == null) this.ultimo = anterior;
        else siguiente.cambiarAnterior(anterior);
    }
}
```

5. Estás implementando la clase Lista. Esta clase ya tiene implementados los métodos básicos: alta, baja, consulta.

Tenés que agregar (implementar) el siguiente método:

```
/*
PRE: agregar
POST: modifica la lista que llama al método: le une lista al final.
*/
public void unir(Lista lista);
```

Nota 1: si necesitás agregar algún otro método que no sea básico, podés hacerlo.

Nota 2: agregá las precondiciones

```
public void unir(ListaT<T> lista){ no usages
    if(this.primerO == null){
        this.primerO = lista.obtenerNodo(pos: 1);
    } else {
        if(lista.obtenerNodo(pos: 1) != null) {
            NodoT actual = this.primerO;
            while (actual.obtenerSiguiente() != null) actual = actual.obtenerSiguiente();
            actual.cambiarSiguiente(lista.obtenerNodo(pos: 1));
        }
    }
    this.cantidad += lista.cantidadElementos();
}
```



```

public void unir(ListaT<T> lista){ no usages
    if(this.primerero != null || lista.obtenerNodo( pos: 1) != null) {
        if(this.primerero == null){
            this.primerero = lista.obtenerNodo( pos: 1);
            this.ultimo = lista.obtenerUltimo();
        } else {
            if(lista.obtenerNodo( pos: 1) != null) {
                this.ultimo.cambiarSiguiete(lista.obtenerNodo( pos: 1));
                this.ultimo = lista.obtenerUltimo();
            }
        }
        this.cantidad += lista.cantidadElementos();
    }
}

```

6. Estás implementando la clase Lista. Esta clase ya tiene implementados los métodos básicos: alta, baja, consulta.

Tenés que agregar (implementar) el siguiente método:

```

/*
PRE: agregar
POST: crea una nueva Lista formada por la unión de la que llama al método
con lista (parámetro), que va al final. Devuelve una referencia a la nueva
lista.
*/
public Lista unir(Lista lista);

```

Nota 1: si necesitás agregar algún otro método que no sea básico podés hacerlo.

Nota 2: agregá las precondiciones

```

public ListaT<T> unir(ListaT<T> lista){ no usages

    ListaT<T> nuevaLista = new ListaT<>();

    NodoT actual = this.primerero;
    while(actual != null){
        nuevaLista.alta(actual.obtenerDato());
        actual = actual.obtenerSiguiete();
    }

    actual = lista.obtenerNodo( pos: 1);
    while(actual != null){
        nuevaLista.alta(actual.obtenerDato());
        actual = actual.obtenerSiguiete();
    }

    return nuevaLista;
}

```

DESARROLLO: USO DE LISTAS

1. La clase Lista ya tiene implementados los métodos básicos: alta, baja, consulta, y tiene una referencia actual con tres métodos: reiniciar(), haySiguiente(), siguiente().

Considerar como implementada la clase Alimento a partir de la siguiente interfaz:

```
public class Alimento {  
    /* Crea un alimento con su nombre, la cantidad de calorías y una lista de  
    los ingredientes que lo conforman */  
    public Alimento (String nombre, int calorías, Lista<String> ingredientes);  
  
    /* devuelve el nombre del alimento */  
    public String obtenerNombre();  
  
    /* devuelve la cantidad de calorías */  
    public int obtenerCalorias ();  
  
    /* devuelve una referencia a la lista de ingredientes */  
    public Lista<string> obtenerIngredientes ();  
};
```

Implementar el método comidasParaCeliacos de la clase BuscadorComidas, según:

```
public class BuscadorComidas {  
    /*  
    Post: busca en "comidas" aquellas que tienen algún ingrediente de la lista  
    "ingredientesPermitidos" y ninguno de la lista "ingredientesNoPermitidos" y  
    que tengan una caloría menor a "caloríaMaxima".  
    Devuelve una referencia a una lista con los alimentos que cumplen con estas  
    características.  
    */  
    public Lista<Alimento> comidasParaCeliacos (  
        Lista<Alimento> comidas,  
        Lista<String> ingredientesPermitidos,  
        Lista<String> ingredientesNoPermitidos,  
        int caloríaMaxima);  
};
```



```
public class BuscadorComidas { no usages
```

```
public ListaConCursorT<Alimento> comidasParaCeliacos(ListaConCursorT<Alimento> comidas, ListaConCursorT<String> ingredientesPermitidos,  
ListaConCursorT<String> ingredientesNoPermitidos, int caloriaMaxima) {
```

```
ListaConCursorT<Alimento> resultado = new ListaConCursorT<>();  
comidas.reiniciar();
```

```
while(comidas.haySiguiente()) {  
Alimento comidaActual = comidas.siguiente();
```

```
if(comidaActual.obtenerCalorias() < caloriaMaxima){
```

```
boolean hayUnoPermitido = false;  
boolean noHayNingunoNoPermitido = true;
```

```
ListaConCursorT<String> ingredientesComida = comidaActual.obtenerIngredientes();  
ingredientesComida.reiniciar();
```

```
while(ingredientesComida.haySiguiente()) {  
String ingrediente = ingredientesComida.siguiente();
```

```
ingredientesPermitidos.reiniciar();  
while(ingredientesPermitidos.haySiguiente()) {  
if(ingrediente.equals(ingredientesPermitidos.siguiente())) {  
hayUnoPermitido = true;  
}  
}
```

```
ingredientesNoPermitidos.reiniciar();  
while(ingredientesNoPermitidos.haySiguiente()) {  
if(ingrediente.equals(ingredientesNoPermitidos.siguiente())) {  
noHayNingunoNoPermitido = false;  
}  
}
```

```
if(hayUnoPermitido && noHayNingunoNoPermitido) resultado.alta(comidaActual);
```

```
}
```

```
}
```

```
return resultado;
```

```
}
```

```
}
```